

Sutradhara: An Intelligent Orchestrator-Engine Co-design for Tool-based Agentic Inference

Anish Biswas
Microsoft Research India
Bengaluru, India

Kanishk Goel
Microsoft Research India
Bengaluru, India

Srirvarshinee S
Microsoft M365 Research
Bengaluru, India

Jayashree Mohan
Microsoft Research India
Bengaluru, India

Alind Khare
Microsoft M365 Research
Bengaluru, India

Anjaly Parayil
Microsoft M365 Research
Bengaluru, India

Chetan Bansal
Microsoft M365 Research
Redmond, USA

Ram Ramjee
Microsoft Research India
Redmond, USA

Abstract

Agentic applications are LLMs that iteratively invoke external tools to accomplish complex tasks [27, 29]. Such tool-based agents are rapidly becoming the dominant paradigm for deploying language models in production. Unlike traditional single-turn inference, agentic workloads chain together multiple LLM calls and tool executions before producing a final response, creating a new performance bottleneck that manifests as increased latency in First Token Rendered (FTR) of the final answer. Through analysis of requests at production scale, we reveal three critical challenges: tool calls account for 30-85% of FTR latency, KV cache hit rates collapse despite substantial context reuse across iterations [9, 30], and sequential orchestration wastes potential intra-request parallelism. These bottlenecks stem from a design gap in which orchestrators and LLM engines operate as decoupled black boxes, preventing cross-layer optimizations.

We present SUTRADHARA, a co-designed agentic inference system that integrates orchestration with LLM serving through a thin API enabling three optimizations: overlap tool execution with subsequent LLM prefill using tool-aware prompt splitting, streaming tool execution to dispatch tools incrementally during decode rather than waiting for complete output, and orchestrator-aware cache management that uses semantic hints to improve hit rates and reduce thrashing. Implemented on vLLM, SUTRADHARA improves the throughput-latency trade-off in agentic systems, sustains up to 77% higher load at the same median FTR latency, or reduces median FTR latency by up to 15% at the same load while reducing end-to-end latency by up-to 11% on A100 GPUs.

1 Introduction

Large language models have rapidly evolved from research prototypes to production systems powering mission-critical applications. While early deployments focused on simple

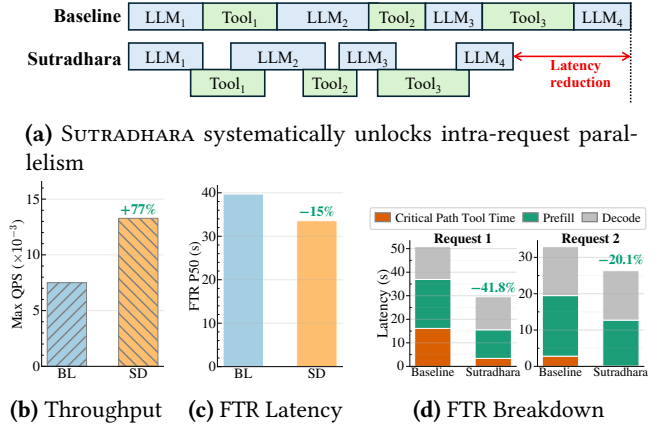


Figure 1. (a) SUTRADHARA systematically parallelizes the execution of LLM and tools and enables workload-aware KV eviction. (b) SUTRADHARA (SD) sustains upto 77% higher load than baseline (BL) at the same median FTR (38s) and (c) reduces median FTR by 15% at the same load. (d) For two random requests in the trace, these techniques reduce FTR by 20 – 42%.

query-response patterns, modern LLM applications increasingly adopt agentic architectures—autonomous systems that iteratively invoke LLMs and external tools to accomplish complex tasks. These systems represent a fundamental shift in how we deploy LLMs. Rather than treating models as stateless oracles that answer isolated queries, agentic architectures enable LLMs to reason, plan, and interact with the external world through iterative tool use [17, 29].

However, this architectural shift introduces a critical performance challenge that existing LLM serving infrastructure fails to address: latency explosions in user-perceived response times. Traditional LLM serving systems optimize for Time-To-First-Token (TTFT) and inter-token latency, treating each inference request as an independent unit of work. In contrast, agentic applications chain together multiple LLM calls and tool invocations in iterative loops before producing a final response. User-perceived latency—measured as First

Token Rendered (FTR) of the final answer encompasses not just a single LLM forward pass, but the cumulative latency of multiple LLM calls interspersed with tool executions until the first user-visible token is generated. In production systems, FTR latency can span seconds or even tens of seconds severely degrading user experience.

To understand these challenges systematically, we conduct the first large-scale empirical study of agentic inference performance using synthetic traces from production workloads in a large cloud provider. Our analysis reveals three critical insights that challenge conventional assumptions about LLM serving bottlenecks. (1) *Tool execution dominates tail latency*, accounting for 30-85% of FTR latency while individual tool calls can exceed LLM prefill time. While LLM prefill and decode have been the traditional focus of optimization efforts [2, 9, 11, 12, 16], we find that tool calls—often dismissed as "thin" external I/O operations—could be a major component of FTR latency. (2) *Sequential orchestration leaves parallelism unexploited*, as 60-80% of each iteration’s prefill is tool-independent yet current systems wait for all tool outputs before beginning the next iteration; similarly tools are executed only after LLM decodes complete, while it is possible to stream tool executions as decode progresses. (3) *KV cache thrashing destroys reuse opportunities* despite context reuse, as workload-agnostic LRU eviction thrashes shared prefixes when agentic requests execute concurrently.

These findings point to a fundamental architectural mismatch: **orchestrators and LLM engines operate as decoupled black boxes**, communicate only through opaque request-response interfaces. The orchestrator has knowledge of iterations, tool dependencies, and prompt composition, while the engine controls scheduling, batching, and KV cache management. Neither layer leverages information from the other to make globally optimal decisions. Breaking this abstraction barrier is essential to address bottlenecks.

We present SUTRADHARA, a co-designed agentic inference system that tightly integrates orchestration logic with LLM engine scheduling through a thin, principled API. SUTRADHARA enables three key optimizations: (1) parallel execution through prompt splitting, where iteration $i + 1$ speculatively begins prefill using tool-independent context while iteration i ’s tools execute, then incrementally extends the prefill when tool outputs arrive; (2) streaming tool dispatch during decode, where a streaming JSON parser identifies complete tool call objects as they are decoded and dispatches them immediately rather than waiting for full decode completion; and (3) orchestrator-aware cache management, where semantic metadata tags and reuse hints guide a priority-aware eviction policy that retains high-value blocks while evicting transient content, combined with request-aware scheduling that prioritizes completing in-flight requests. Figure 1 shows how intra request parallelism enables SUTRADHARA to achieve significant latency reduction of up-to 42% per request.

We implement SUTRADHARA as an asynchronous event-driven orchestrator built on top of vLLM v0.11.0, with targeted modifications to the v1 scheduler totaling fewer than 3500 lines of code. The orchestrator-engine interface consists of five new API calls (Table 1) that enable hint passing, parallel execution coordination, and cache metadata communication. Our implementation requires no changes to model architectures, training procedures, or inference kernels. We evaluate SUTRADHARA across A100-80GB GPUs using synthetic traces from production on Qwen3-14B. Compared to the vLLM baseline, SUTRADHARA achieves a better throughput-latency tradeoff, sustains up to 77% higher load at the same median FTR latency, or reduces median FTR latency by up to 15% at the same load while improving end-to-end latency by 11%.

In summary, this paper makes the following contributions:

- **Empirical characterization.** First large-scale analysis of agentic inference workloads, identifying tool execution variance, sequential bottlenecks, and cache thrashing as dominant sources of tail FTR latency (§3).
- **Co-designed architecture.** We propose SUTRADHARA, an orchestrator-engine co-design, with three novel techniques; prompt splitting with tool execution overlap, streaming tool dispatch, and semantic cache management that exploits agentic request structure (§4).
- **Implementation and evaluation.** We implement SUTRADHARA on vLLM with minimal invasiveness and demonstrate across models and workloads that it sustains up to 77% higher load at the same median FTR latency, or alternatively reduces median and tail FTR latency by up to 15% and 11% at the same load on an A100-GPU cluster (§5).

2 Background

2.1 Agentic Inference

Agentic inference represents a fundamental departure from traditional single-turn LLM inference. In standard LLM serving, a user submits a query, the model performs a single forward pass (prefill followed by autoregressive decode), and returns a complete response. The serving system optimizes for metrics like Time-To-First-Token (TTFT) and per-token decode latency, treating each request as an isolated, stateless computation. Agentic inference, by contrast, enables LLMs to interact with external tools and APIs through iterative execution loops. Rather than producing a final answer in one shot, the LLM reasons about a task, decides which tools to invoke, examines their outputs, and continues this process until reaching a satisfactory answer.

2.2 Agentic inference serving framework

Figure 2 illustrates the standard architecture for agentic inference serving, consisting of three primary components that interact through well-defined interfaces.

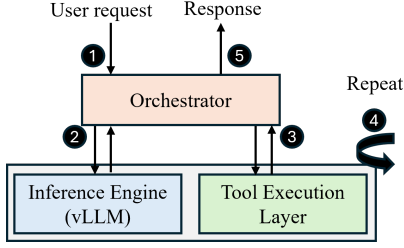


Figure 2. Workflow: (1) User request arrives; (2) Orchestrator sends LLM query and receives response; (3) Tools optionally invoked based on response; (4) Iterative loop of LLM and tools; (5) Final user-visible response returned.

LLM Serving Engine. The engine handles low-level inference execution—batching requests, managing GPU memory, scheduling prefill and decode operations. Modern production systems use frameworks like vLLM [9] or SGLang [30], which implement optimizations such as continuous batching, PagedAttention, and prefix caching. The engine exposes a request-response API where clients submit prompts and receive generated token sequences. Importantly, the engine treats each LLM call as independent—it has no visibility into whether a request is part of an agentic workflow, or which iteration it belongs to.

Orchestrator. The orchestrator implements the control logic for agentic execution. Built as an asynchronous event-driven system (typically using Python’s asyncio or similar frameworks), it manages the iterative loop: constructing prompts from conversation history and tool outputs, dispatching LLM calls to the engine, parsing structured outputs to extract tool specifications, invoking tools through their respective APIs, and tracking iteration state. The orchestrator maintains semantic knowledge about request structure—which prompt sections depend on tool outputs, which tools are executing, and how context accumulates across iterations. However, this knowledge remains isolated within the orchestrator and is not communicated to the engine.

Tool Execution Layer. Tools are services accessed through APIs, libraries, or sand-boxed execution environments [23]. Common tools include web search, code execution, file system operations, and API calls to third-party services. Tools execute asynchronously and independently; when multiple tools are invoked in the same iteration, they run in parallel [8]. Tool execution time varies dramatically based on query complexity, external service load, and network conditions, ranging from milliseconds to seconds.

Multi-iteration execution structure. An agentic request consists of multiple iterations, where each iteration i follows a structured pattern:

1. **LLM Call:** The orchestrator submits a prompt to the LLM engine containing system instructions, conversation history, and (for $i > 1$) outputs from previous

tool calls. The engine performs prefill to process the prompt, then generates tokens auto-regressively during decode. For the intermediate iterations, the decode output specifies which tools to invoke, typically formatted as structured JSON. For the final iteration, the decode output is the user-facing final response.

2. **Tool execution :** The orchestrator parses the tool specifications and dispatches each tool for execution. Each iteration can dispatch multiple tools, that may execute independently and finish at different times.

Sequential execution. Current agentic systems enforce a strict sequential execution between prefills, decodes, and tool calls per iteration. Our analysis shows that there is scope to enable intra-request parallelism across iterations, thereby optimizing request completion latencies.

3 Analysis

This section presents the agentic workload characterization that highlights bottlenecks in orchestration and KV cache behavior. We conclude with design requirements for an optimized system.

3.1 Trace collection

Agentic Platform. To understand the performance characteristics of agentic inference in production environments, we analyze workloads from a major cloud provider’s internal agentic platform deployed across thousands of enterprise customers. This platform provides a general-purpose orchestration framework that manages iterative LLM-tool execution loops. It dispatches requests to a heterogeneous fleet of LLM serving endpoints and coordinates calls to an extensible tool registry containing 20+ integrated services including web search APIs, enterprise chat, email, file searches, code execution, internal knowledge bases, and third-party SaaS integrations. The orchestrator implements a standard agentic execution pattern: issue an LLM request with available tools, parse structured output to identify tool calls, execute tools and collect outputs, append results to context, and iterate until the LLM produces a final response. The orchestrator could modify instructions and system prompts in any iteration depending on the set of tool calls made. This architecture is representative of popular agentic systems including LangChain [10], AutoGen [18], and proprietary enterprise frameworks.

Workload Generation. We analyze workloads issued on this platform with synthetic user profiles and data, instead of the real customer-facing queries as its prompt tokens are eyes-off for privacy reasons. The synthetic queries on this actual production platform are created through an automated pipeline that first provisions test users along with their synthetically generated enterprise context, including files, chats, and calendar events. Then, a structured sets of

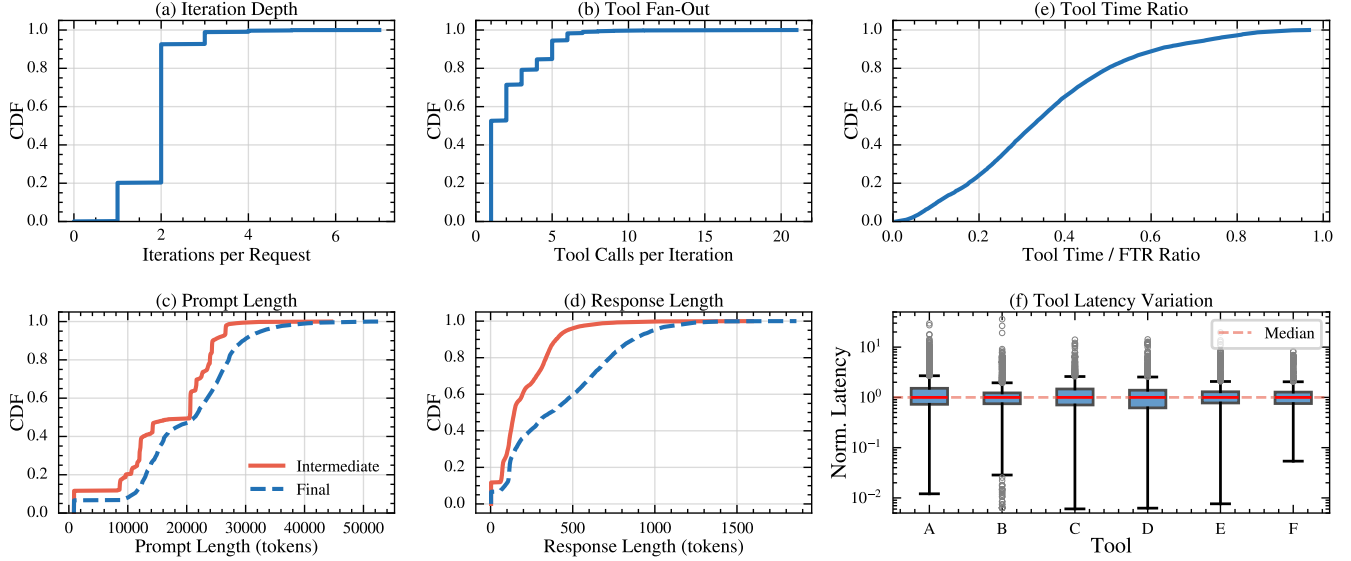


Figure 3. Detailed statistics of the agentic production trace. The distributions illustrate the structural characteristics of multi-step requests (a-d) and the associated tool execution dynamics (e-f). (e) denotes the normalized tool latency with respect to the median.

evaluation queries are executed on the platform, each designed to replicate realistic tasks such as document retrieval, summarization, and information search. By modeling workflows and entity relationships, the synthetic workload environment closely mirrors production scenarios. We also validate all the observations gained from synthetic workloads against customer-facing production workload. We analyze 6000 agentic requests across different user categories.

3.1.1 Metrics and Definitions. The key metrics are:

First Token Rendered (FTR). FTR is defined as the elapsed time from user request submission to the rendering of the *first token* of the final user-visible response.

Iteration Depth. No. of LLM calls in an agentic request.

Tool Call Fan-Out. Number of tool invocations *per iteration*.

3.2 Trace statistics

We first present the overall trace characteristics. For each request, we categorize the number of LLM iterations into two types; intermediate iterations that perform tool invocations and a final iteration that generates the user visible response and doesn’t result in any tool calls. Figure 3 presents the overall trace characteristics (a) the distribution of iteration depth per request (b) tool call fan out per iteration, (c) prefill length distribution and (d) generated token distribution by iteration type. A median agentic request issues about 2 LLM iterations, one intermediate and one final, while in the tail, the iteration depth goes up to 7. Similarly, the per iteration tool call fan out is 2 in the median case, but some iterations could have as high as 21 tool calls made in a single iteration. The median prompt length across intermediate and final iterations is about 20K tokens, while the tail could be 3x

higher. The majority of the context is the system prompt (that could vary depending on the iteration and tools invoked in the previous iteration), and tool-specific instructions and formatting metadata. However, the median generated tokens for intermediate iterations is about 5x lower than the final iteration; this makes the intermediate iterations more prefill-bound, while the final iteration is decode-bound.

3.3 Trace-driven analysis

We present our key findings on the production workload.

Finding 1 : Tool call execution dominates the tail FTR.

Figure 3(e) presents the cumulative distribution of tool execution time as a fraction of total FTR latency across our workload. While the median request spends only 32% of its FTR latency executing tools (with the remaining 68% spent in LLM prefill and decode), the tail exhibits dramatically different behavior. At the 90th percentile, tool execution accounts for 61% of FTR latency, climbing to 85% at the 99th percentile. This tail dominance contradicts the conventional assumption that tool calls are lightweight I/O operations contributing marginally to end-to-end latency [11, 16].

Our analysis further reveals variability in tool execution latency. Figure 3(f) shows box plots of tool latencies normalized to each tool’s median (p50) for six representative production tools. Even after normalization, all tools exhibit wide dispersion and pronounced right tails. By p75, normalized latency reaches $1.23\text{--}1.52 \times p50$ across tools, and at p90 spans $1.60\text{--}3.28 \times p50$. This heavy-tailed behavior indicates that tool execution latency remains highly sensitive to factors such as query complexity, backend contention, and the volume of data accessed (e.g., in search-style tools). As a result, tool execution time remains inherently difficult

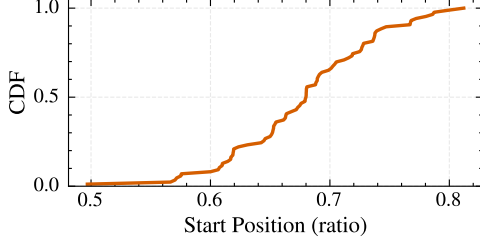


Figure 4. CDF of prompt prefix fraction independent of tool output

to predict, making tool-execution-time-aware KV caching strategies impractical in practice [11].

The magnitude of tool execution time presents a clear optimization opportunity. Even partially overlapping tool execution with subsequent LLM computation could yield significant latency reductions. However, exploiting this opportunity requires tight coordination between the orchestrator and LLM engine, which is a capability fundamentally absent in current black-box architectures where components communicate solely through request-response interfaces.

Finding 2 : Sequential orchestration leaves substantial parallelism unexploited. Current orchestrators enforce strict sequential execution within each agentic request: iteration i completes LLM decode, all tool calls execute to completion, all tool outputs return, and only then does iteration $i + 1$ begin prefill. This pipeline unnecessarily serializes the three phases – prefill, decode, and tool execution that could potentially overlap. We identify two specific sources of unexploited parallelism that stem from this rigid sequencing.

Opportunity 1: Prefill-tool overlap. Figure 4 analyzes prompt composition across iterations, measuring the fraction of each prompt consisting of tool-independent content versus tool-dependent outputs. We observe that 50–80% of iteration $i + 1$ ’s prompt is available when iteration i finishes decode. System instructions, conversation history, and templates don’t depend on tool outputs. Only the final 20–50% needs tool results. This creates a natural split point. The orchestrator could start prefill on the part of the prompt independent of tool outputs while tools execute, and extend the prefill incrementally.

Opportunity 2 : Decode-tool overlap. LLMs generate tool calls as JSON: ["tool": "search", "query": "...", "tool": "plot", "query": "..."]. Current orchestrators wait for the entire array before dispatching tools. But once the first JSON object completes (closing `}}`), that tool can execute immediately, allowing for streaming tool execution as decodes progress.

Finding 3 : KV cache thrashing lowers prefix reuse. Agentic requests exhibit significant context reuse both across iterations within a request, as well as across requests that make similar tool calls. Across iterations, conversation history accumulates, whereas system prompts and instruction templates to consume tool output depend on tool call made in the prior iteration; such system prompts could be shared

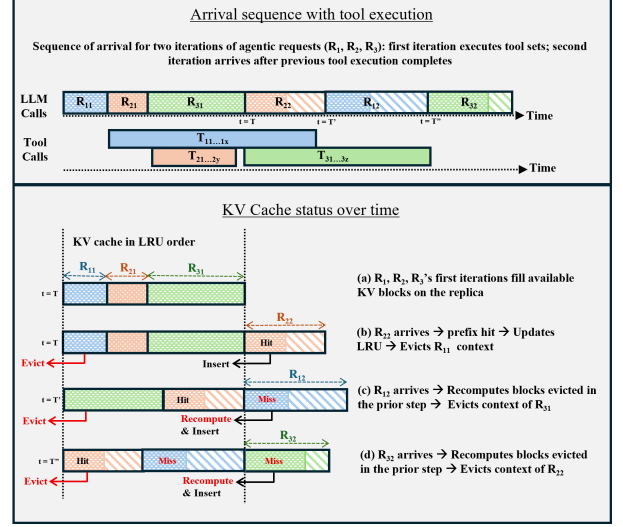


Figure 5. Thrashing due to workload-agnostic KV eviction policy

across requests that make similar tool calls. Standard prefix caching mechanisms should enable high cache hit rates by reusing these shared prefixes. However, we observe systematic cache thrashing that severely degrades performance under concurrent agentic workload execution.

Root cause: Workload-agnostic LRU eviction. Popular serving platforms [9, 30] employ Least Recently Used (LRU) eviction policies that treat all cached KV blocks identically, lacking awareness of agentic request structure. To understand the thrashing behavior, consider three concurrent agentic requests R_1 , R_2 , and R_3 shown in Figure 5. Each request performs two iterations, where the first iteration generates tool calls (denoted $T_{i1..1x}$, $T_{i1..2y}$, $T_{i1..3z}$ for requests R_i) and the second iteration can only begin after the previous iteration’s tool execution completes.

At time $t = T$, all three requests complete their first LLM calls (R_{11} , R_{21} , R_{31}) and their KV blocks fill the available cache capacity in LRU order (Figure 5(a)). When R_2 ’s second iteration (R_{22}) arrives at $t = T'$, it gets a prefix cache hit from R_{21} . However, inserting R_{22} ’s new KV blocks causes LRU to evict R_{11} ’s context (Figure 5(b)). Subsequently, when R_{12} arrives at $t = T''$, it must recompute the evicted blocks from R_{11} , which in turn triggers eviction of R_{31} ’s context (Figure 5(c)). This cascading pattern continues as R_{32} arrives at $t = T'''$ and must recompute R_{31} ’s evicted blocks, further evicting R_{22} ’s context (Figure 5(d)). The problem is that LRU evicts by recency, ignoring that R_{11} , R_{21} , R_{31} are first-iteration contexts that will be reused by blocked second iterations. We address this by using workload-aware eviction hints to evict KV-cache regions with lower reuse likelihood (§4.3).

In this work, we assume only HBM caching; thus, evicted KV blocks are recomputed. Alternatively, these blocks could be offloaded to secondary storage (e.g., CPU) using systems such as LMCache [5] or Mooncake [24]. In that case, our

API Call	Purpose
<code>submit_partial_prefill()</code>	Submit tool-independent prompt slice
<code>extend_prefill()</code>	Append tool outputs to pinned partial prefill context
<code>register_streaming_callback()</code>	Receive partial decode outputs token-by-token
<code>tag_kv_blocks()</code>	Annotate cached KV blocks with semantic hints (e.g., <code>system_prompt</code> , <code>response</code>)
<code>set_reuse_priority()</code>	Set priorities among KV blocks for pinning

Table 1. APIs that enable orchestrator-engine co-design.

ideas remain applicable, as prefetching the right KV blocks is still required due to the HBM-storage latency hierarchy.

3.3.1 Summary. Our three key findings reveal a fundamental architectural problem: current agentic systems suffer from a **lack of co-design between the orchestrator and LLM inference engine**. Today’s architectures treat these components as independent black boxes that communicate solely through opaque request-response interfaces. The orchestrator dispatches individual LLM calls and waits for complete responses before proceeding, while the engine treats each request as an isolated inference job with no awareness of its role in a multi-iteration agentic workflow. This decoupling prevents both layers from exploiting information that could enable critical optimizations.

4 SUTRADHARA: Design and Implementation

SUTRADHARA extends the standard LLM serving architecture with a thin coordination layer that enables the orchestrator to communicate semantic hints about agentic request structure to the engine. As before, the orchestrator maintains knowledge of iteration boundaries, prompt composition, and tool dependencies, while the engine controls scheduling, batching, and KV cache management. However, through five new API calls shown in Table 1, the orchestrator guides engine decisions without requiring model-level modifications.

4.1 Parallel execution via prompt splitting

Our analysis revealed that 50-80% of iteration $i+1$ ’s prompt is available immediately when iteration i completes decode, yet current systems wait for all tool outputs before beginning any prefill computation. This sequential execution leaves parallelism unexploited. However, naively starting prefill before tool outputs arrive introduces two challenges: (1) correctness: the engine must know where to splice tool outputs into the partial prompt, and (2) efficiency: the engine must retain the prefilled KV cache while tools execute, potentially without premature eviction.

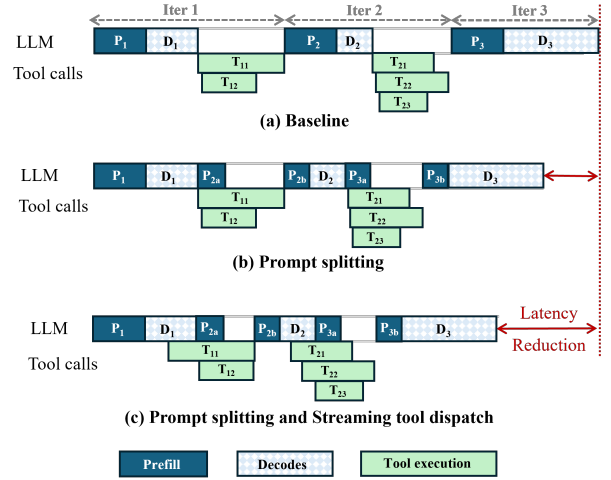


Figure 6. Intra-request parallel execution in SUTRADHARA

Figure 6 illustrates our approach across three iterations of an agentic request. Each iteration consists of prefill (P_i) and decodes (D_i), followed by tool executions (T_{ij} for tool j in iteration i). Figure 6a is the baseline sequential execution done by systems today; iteration 1 completes decodes D_1 , all the tool executions T_{11} , T_{12} and then iteration 2 begins prefill P_2 . This pattern repeats for subsequent iterations. The LLM engine schedules iterations from other agentic requests in the idle time between tool calls to keep GPU occupied, creating long intra-request sequential chains of execution.

Prompt splitting as demonstrated in Figure 6b breaks this sequential dependency by partitioning prompts into tool-independent and tool-dependent slices:

- 1. Slice identification:** When the iteration i completes decodes and generates tool calls, the orchestrator aware of prompt template, identifies the insertion point where tool outputs will be spliced, typically between system instructions/history and the tool results section.
- 2. Eager prefill execution:** The orchestrator submits the tool-independent prefix P_{2a} using `submit_partial_prefill()`, which returns a continuation handle. The engine computes prefill while tools execute concurrently. The KV cache blocks from partial prefill are tagged with high priority via `set_reuse_priority()` to prevent eviction.
- 3. Prompt extension:** Once tool outputs arrive, the orchestrator constructs the tool-dependent suffix and calls `extend_prefill()` with the continuation handle. The engine splices the new content (P_{2b}) onto the pinned KV cache from P_{2a} , completes the prefill, and proceeds to decode D_2 . If tool execution fails or times out, the orchestrator discards the partial prefill and sets appropriate hints for its KV for the engine to release pinned resources.

This approach overlaps tool execution with a part of prefill, reducing the request’s end-to-end latency.

4.2 Streaming tool dispatch with decodes

Agentic systems generate tool calls as structured JSON arrays during the LLM decode phase. Figure 6c shows that current systems wait for complete decode output before parsing and dispatching tools, introducing unnecessary serialization. However, once a complete tool invocation structure is decoded (e.g., the closing `}` of `"tool": "search", "query": "..."`), that tool can execute immediately.

SUTRADHARA implements streaming tool dispatch through token-level callbacks from the engine to the orchestrator. When submitting a decode request that will generate tool calls (intermediate iterations), the orchestrator calls `register_streaming_callback()` with a handler function. The engine invokes this handler after each decoded token. The orchestrator maintains a streaming JSON parser that accumulates tokens and identifies complete objects. When a tool call object closes (final `}` token), the parser extracts the tool name and parameters. As soon as a complete tool invocation is identified, the orchestrator dispatches it for execution without waiting for remaining decode tokens. Subsequent tool calls in the JSON array are dispatched as they complete. The orchestrator tracks completion of all tools before proceeding to the next iteration.

4.3 Workload-aware KV cache management and scheduling

Our analysis identified systematic KV cache thrashing in concurrent agentic workloads despite significant context reuse. Figure 5 illustrates the root cause: workload-agnostic LRU eviction treats all cached blocks identically, causing cascading evictions that destroy reuse opportunities. LRU evicts based solely on recency, ignoring workload structure. It cannot distinguish between low-value blocks (transient content unlikely to be reused) and high-value blocks (first-iteration contexts that will be reused by second iterations currently blocked on tool execution). In SUTRADHARA, we use semantic tagging and priority-based eviction to tackle this.

KV block tagging. When the orchestrator submits LLM requests, it tags KV cache blocks with semantic metadata using `tag_kv_blocks()`.

- **System prompts:** Marked as `SYSTEM_PROMPT` with high reuse priority. Depending on the iteration type (intermediate or final) and the combination of tool calls made in the previous iteration, there could be many variants of system prompts that the orchestrator constructs using predefined rules; for e.g., the system prompt could contain instructions on how the output of the tool call made in the previous iteration should be consumed and interpreted. Such system prompts occur before the user-specific query and hence could be shared across many requests.

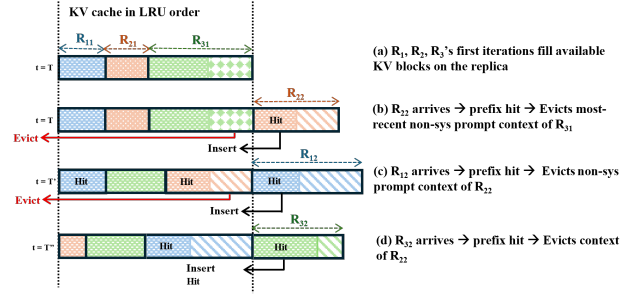


Figure 7. Workload-aware KV eviction policy

- **User specific query:** Marked as `USER_QUERY` and contains the request-specific context including user information which can result in prefix matches intra-request alone; across iterations of the same agentic request.
- **Tool outputs:** tagged as `TOOL_OUTPUT_ITER_i`, these are reused by subsequent intermediate iterations in an agentic request, provided that these iterations share the system prompt. If the system prompt in the next iteration changes due to a new tool call in the prior iteration, the prefilled tool output context from a prior iteration becomes useless as it is appended after the system prompt and user query.
- **Final response:** Tagged as `RESPONSE`. The final LLM call's decodes are the user-facing output tokens; these have no reuse potential and are candidates for eviction.
- **Partial prefills:** These are KV blocks from `submit_partial_prefill()`, tagged as `PARTIAL_PREFILL` with maximum priority until its extension prompt is completed.

Priority-based eviction policy. Using the tags on the KV blocks, the engine's scheduler implements a priority-aware eviction policy that respects orchestrator hints. When cache capacity is exceeded, the scheduler evicts the lowest-priority blocks first. Within a priority tier, LRU ordering is used as a tiebreaker. The eviction priority (evicted first) $\rightarrow$ (evicted last) is as follows:

RESPONSE $\rightarrow$ TOOL_OUTPUT $\rightarrow$ USER_QUERY
 $\rightarrow$ SYSTEM_PROMPT $\rightarrow$ PARTIAL_PREFILL

Example. Returning to the scenario in Figure 5, our policy prevents cascading evictions as shown in Figure 7. At time $t = T'$, when R_{22} needs to insert new blocks, the eviction policy targets low-priority blocks (tool outputs) rather than R_{11} 's context, which is tagged with high reuse priority since R_{11} 's tools are executing. When R_{12} arrives at $t = T''$, it finds R_{11} 's blocks still cached, achieving a prefix hit and avoiding recomputation. Similarly, R_{31} survives until R_{32} arrives, preventing the cache miss chain.

Workload-aware scheduling. While scheduling is orthogonal to SUTRADHARA's optimizations, it remains a critical component for efficient agentic systems. Although agentic requests typically issue multiple LLM calls, the existing serving engines schedule at the granularity of individual LLM

calls (e.g., FIFO), without global agentic context. As a result, requests that issue LLM calls more frequently may be unfairly prioritized over earlier-arriving agentic requests. To address this, SUTRADHARA enforces a workload-aware policy in the serving engines that preserves a global FIFO ordering of LLM calls with respect to the arrival time of agentic requests in the orchestration, ensuring fairness. Integrating more sophisticated scheduling techniques designed to prevent starvation, such as Autellix[16], is left for future work.

4.4 Implementation

We implement SUTRADHARA using $\approx 3,500$ lines of Python consisting of a serving engine layer and an independent orchestrator. We build the serving engine on top of vLLM (v0.11.0) and enable chunked prefill with prefix caching by default. For the orchestrator, we developed a lightweight, asyncio-based event-driven framework designed to accurately replay synthetic production traces. SUTRADHARA’s modular design readily supports integration with alternative serving backends [3, 20]. Our engine modifications natively embed workload semantics, allowing the orchestrator to leverage this contextual metadata. For robustness, the components rely on a heartbeat-based membership protocol.

5 Evaluation

Our evaluation answers the following questions:

1. Does SUTRADHARA achieve a better throughput-latency trade-off for FTR and E2E latency on production agentic workloads?
2. Can SUTRADHARA generalize to disaggregated serving and diverse LLMs?
3. How does SUTRADHARA perform on open source agentic datasets?
4. What is the independent contribution of each optimization in SUTRADHARA?
5. Does SUTRADHARA increase KV cache hit rates?

5.1 Experimental Setup

Models. We evaluate SUTRADHARA on Qwen3-14B [6], a 14B-parameter open-source model featuring native function-calling support. At BF16 precision, this model fits within the memory budget of a single A100-80GB GPU supporting a 128K context window and accommodates the maximum prompt lengths observed in our workloads. SUTRADHARA’s optimizations operate at the orchestrator-engine interface independent of the models or kernels used in the engine. To validate this, we evaluate SUTRADHARA on Gemma-27B in §5.3.

Workloads. We evaluate SUTRADHARA across production and open-source traces.

Production Trace. It uses a subset of the production trace detailed in §3. We pick a stratified sample of 120 requests from the production trace, characterized by a high degree of tool-call fan-outs per request. We replay it using the original prompts, recorded tool outputs, and the decode lengths extracted from the trace. Tool-call latencies are normalized according to the observed tool-to- LLM ratio within the original traces as described in §3. Additionally, we present the evaluation of SUTRADHARA on a different subset of production traces with median tool-call fan-out in Appendix and observe similar magnitude of gains. Request arrival times follow a Poisson process. We report the mean success metrics with confidence bands across five seeds.

Open-Source Traces. The traces are derived from the following open-source benchmarks—

- **BFCL v4 Web Search** [22]: An open-source agentic benchmark that evaluates multi-hop web search via function-calling agents with standardized tools. We use 56 single-turn requests randomly sampled from an initial set of 100, each containing at least one multi-tool invocation. Agent trajectories are short, with a mean of 4.23 iterations and low tool fan-out per iteration (≈ 2). Tool execution dominates latency, averaging 1.09 s per call (variance 1.7 s) and accounting for approximately 40% of E2E request time.
- **SWE-Bench** [7]: An open-source agentic software engineering benchmark that captures long-horizon code exploration, execution, and repair over real-world GitHub issues. We evaluate 60 single-turn requests randomly sampled from a pool of 500, exhibiting substantially longer agent trajectories (mean 20.0 iterations) with similarly low tool fan-out (≈ 2). Tool calls are shorter on average (0.29 s, variance 1.14 s) and contribute roughly 30% of overall execution time.

Both traces follow a strictly append-only prompt structure, where each iteration appends new context without modifying prior prompt prefixes. Most evaluations use the production trace, unless explicitly stated.

Hardware. We evaluate SUTRADHARA on A100-80GB GPUs [19]. For colocated experiments, we use a single A100. For disaggregated experiments, we use two A100-80GB GPUs - one dedicated prefill node and one dedicated decode node - connected over NVLink.

Baselines. We compare against a baseline using vLLM as the serving backend with Prefill-Decode co-location at a chunk size of 256, utilizing request-aware scheduling that maintains FIFO ordering at an agentic request level instead of individual LLM calls. SUTRADHARA builds on this baseline and additionally introduces prompt splitting, streaming dispatch, and orchestrator-aware KV cache management. We also compare against the PD disaggregated baseline in vLLM. Automatic prefix caching is enabled by default. We also compare against Continuum [11], a concurrent work that optimizes agentic systems by pinning KV cache blocks

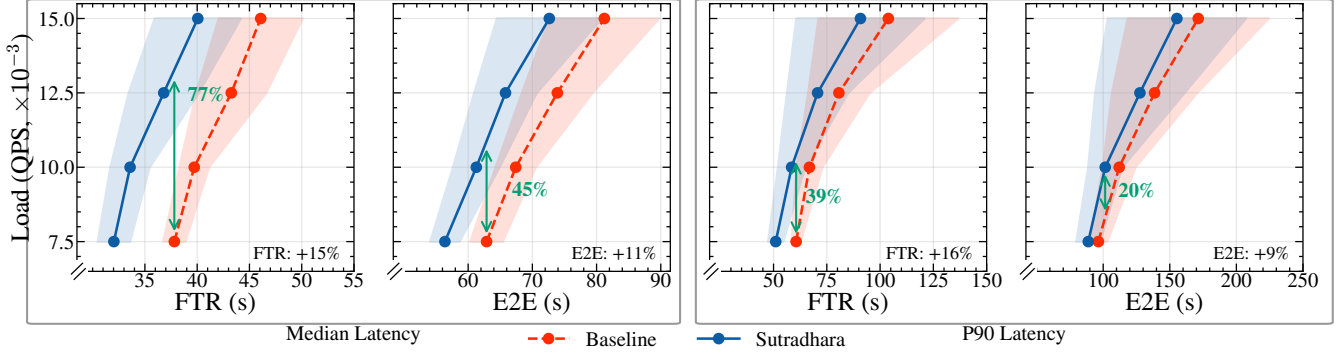


Figure 8. Serving capacity curves for SUTRADHARA and Baseline with p50/p90 FTR and E2E metrics on x-axis and ingest load (QPS) on y-axis. Curves that lie closer to the top-left are better: higher load sustained at lower latency. SUTRADHARA is closer to the top-left with a better throughput-latency tradeoff compared to baseline: sustains upto 77% higher load at the same latency or achieves upto 15% reduction in p50 FTR latency at the same load. Similar gains extend to E2E latency.

during tool execution using a time-to-live mechanism to prevent eviction-induced recomputation across iterations. We integrate Continuum with our orchestrator on a best-effort basis and set the TTL to 6s, based on mean tool call time in our production trace.

Metrics. We evaluate SUTRADHARA on two primary latency metrics. *First Token Rendered* (FTR) measures the time from the initial user request submission to the generation of the first token in the final user-visible response. *End-to-End* (E2E) latency reflects the total request completion time.

5.2 End-to-End Performance

We assess whether SUTRADHARA achieves a better throughput-latency tradeoff compared to baselines. Specifically, we evaluate SUTRADHARA on varying loads (0.0075, 0.01, 0.0125 and 0.0015 QPS) and measure p50/p90 FTR and E2E latency. To characterize this trade-off, we plot serving capacity curves with ingest load (QPS) on the y-axis and latency on the x-axis. A system exhibits a better throughput-latency trade-off if it can sustain higher load at lower latency, or equivalently, achieve lower latency at the same load—corresponding to curves that lie closer to the top-left of the plot.

Figure 8 shows the serving capacity curves for both SUTRADHARA and baseline. SUTRADHARA achieves a better throughput-latency tradeoff compared to baseline, with its service capacity curve lying to the left of the baseline. At the same p50 FTR latency, SUTRADHARA sustains up to 77% higher load; conversely, for a fixed load, it achieves up to a 15% reduction in p50 FTR latency. SUTRADHARA also delivers improved trade-offs for tail latency. For the same p90 FTR latency, it sustains up to 45% higher load at the same latency, or achieves up to an 11% reduction in tail FTR latency at the same load. Similar benefits extend to end-to-end (E2E) latency, with up to a 9% reduction in p90 E2E latency. SUTRADHARA achieves a better trade-off as it unlocks intra-request

parallelism through its co-design and hides tool call time under prefills and decodes.

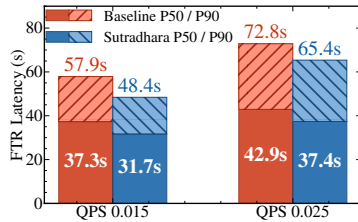
The gains are more pronounced for FTR than for E2E latency because the E2E metric includes the decode phase of the final iteration, which does not trigger any subsequent tool execution. As a result, this phase cannot benefit from prompt splitting or streaming dispatch optimizations, limiting the achievable acceleration in E2E latency. To understand these gains, we show detailed FTR and E2E latency CDFs across varying ingest loads in Appendix (Figure A.1, A.2).

5.3 Generalizability

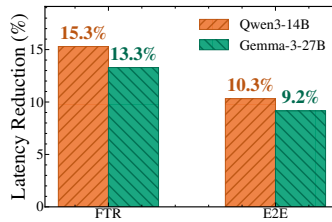
Disaggregated Serving. We evaluate SUTRADHARA in disaggregated prefill-decode setting to assess if SUTRADHARA’s generalizes to a different deployment. We run these workloads at 0.015 and 0.025 QPS, maintaining a per-GPU load equivalent to our colocated experiments. Figure 9 (a) summarizes the FTR and E2E latency improvements achieved under disaggregated serving. Across both evaluated load levels, SUTRADHARA consistently reduces both median and P90 FTR by 12% to 16%. These gains are consistent with the colocated results, demonstrating that SUTRADHARA’s optimizations transfer across deployment topologies without modification.

Model Generalizability. SUTRADHARA’s optimizations operate entirely at the orchestrator-engine interface and universally apply to tool-calling LLMs served through vLLM. To validate this architecture-agnostic claim, we evaluate our system using Gemma-3-27B [26]: a more parameter-heavy model with a different attention mechanism than Qwen3-14B.

Figure 9 (b) shows the median FTR and E2E improvements for both models at 0.0075 QPS. SUTRADHARA reduces median FTR by 13.3% and E2E latency by 9.2% on Gemma-3-27B. This demonstrates SUTRADHARA’s latency improvements generalize across different models. The performance difference



(a) Disaggregated serving



(b) Model generalizability

	QPS	BL (s)	SD (s)	Gain
BFCL v4	0.50	20.37	18.91	7.2%
	1.50	20.87	18.79	10.0%
	2.00	21.59	19.87	8.0%
SWE Agent	0.05	48.47	42.10	13.2%
	0.10	51.95	45.20	13.0%
	0.25	61.86	56.77	8.2%

(c) Median FTR on open source traces

Figure 9. SUTRADHARA latency gains generalize to (a) disaggregated serving, (b) different models (Gemma3-27B), and (c) open source traces (BL = Baseline and SD = Sutradhara)

between the two models directly stems from Gemma-3-27B’s higher LLM compute requirements. Specifically, its prefill and decode phases are 1.34× and 1.60× slower, respectively. This increased compute time reduces the tool fraction of the overall FTR from 12.9% down to 9.0% limiting the available time window for SUTRADHARA’s optimizations to take effect.

Open Source Traces. We evaluate SUTRADHARA on the BFCL Web Search and SWE-Bench traces using the Qwen3-14B model. Table 9 (c) shows the results across varying load levels. The baseline saturates at different load levels for both traces (2 QPS for BFCL and 0.25 QPS for SWE-Bench). SUTRADHARA’s optimizations generalize and reduce median FTR for both the traces. SUTRADHARA causes 10% and 13.2% p50 FTR reduction at 1.5 QPS load on BFCL, and 0.05 QPS load on SWE-Bench respectively. The lower median improvement observed on the open-source traces relative to the production trace (7.2-13.2% versus 15.3%) is primarily due to two key differences, the BFCL and SWE-Bench traces have a) less tool call fan-out that limits the opportunities to overlap tool calls with the decode, and b) strictly append-only prompt structure that limits gains from prefill splits as the splitted prefill may already remain prefix cached (from previous iteration’s prefill) leading to less overlap with tool calls.

5.4 Ablation Study

5.4.1 Cumulative Impact of Optimizations. Table 2 isolates the contribution of each optimization at 0.0075 QPS by incrementally adding them.

Prompt Splitting. Prompt splitting reduces median FTR by 6.1% and E2E latency by 3.5%. Although 50–80% of the prompt is available for the split (§3.3), these time savings are bounded the extent of overlap between prefill and tools calls. For instance, whenever a tool executes faster than the partial prefill, the remaining prefill execution still remains on the critical path. Moreover, E2E latency gains remain less compared to FTR as decode dominates E2E latency and this optimization only performs prefill overlaps with tool calls.

Streaming Dispatch. Streaming dispatch further contributes to latency reduction by overlapping decodes with tool calls. It achieves an 8.3% FTR and 5.5% E2E latency improvement

Table 2. Each optimization’s contribution at 0.0075 QPS. The median latency averaged over 3 seeds. PS: prompt splitting, DS: streaming dispatch, KV: cache management. Improvements are relative to Baseline+Sched.

Config	FTR (s)	E2E (s)	Cumulative		Incremental	
			FTR	E2E	FTR	E2E
Baseline	37.45	61.57	—	—	—	—
+PS	35.17	59.39	+6.1%	+3.5%	+6.1%	+3.5%
+PS+DS	32.07	56.03	+14.4%	+9.0%	+8.3%	+5.5%
+PS+DS+KV	31.39	54.93	+16.2%	+10.8%	+1.8%	+1.8%

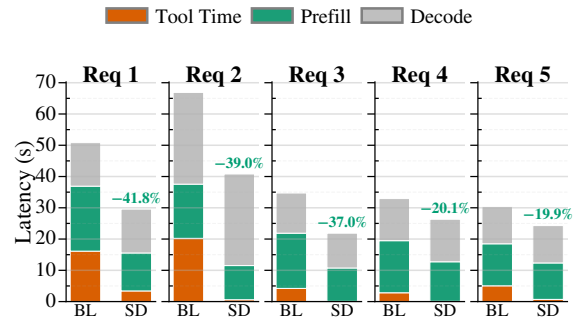


Figure 10. FTR Latency breakdown across five representative requests. B is baseline, SD is SUTRADHARA

on top of prompt splitting, yielding cumulative improvements of 14.4% and 9.0%, respectively. Especially for high tool call fan-out requests, this optimization converts a purely sequential decode-then-dispatch pipeline into overlapped execution reducing the critical path.

KV Cache Management. This optimization further contributes a 1.8% improvement in both FTR and E2E latency. This reflects a direct reduction in prefill recomputation driven by higher cache hit rates under concurrent agentic workloads. as demonstrated in §5.4.3. However, these increased hit rates don’t proportionally translate to similar latency gains as the decode dominates the E2E latency.

5.4.2 Per-Request FTR Latency Breakdown. To isolate the sources of performance improvement, Figure 10 decomposes the FTR latency of five randomly selected tool-heavy

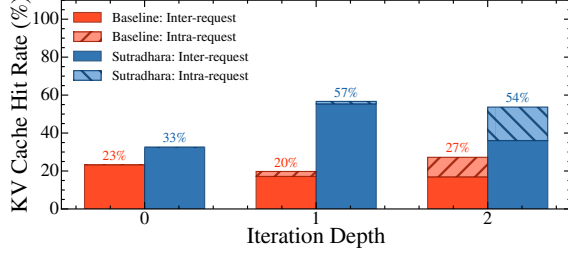


Figure 11. SUTRADHARA vs baseline on inter- vs intra-request cache hit rate by agentic requests’ iteration depth.

requests into three distinct phases: critical path tool time, total prefill time, and total decode time summed across intermediate iterations of each request. The breakdown reveals two key advantages. First, SUTRADHARA reduces the critical path tool time which is a direct consequence of streaming tool executions concurrently with the decode. Second, SUTRADHARA decreases the total prefill time compared to the baseline. This is due to higher cache hit rates (§5.4.3) and overlapping prefill execution with the tool invocations. The breakdown reveals how SUTRADHARA reduces FTR latency and enables better throughput-latency trade-offs (§5.2).

5.4.3 KV Cache Analysis. Figure 11 illustrates the decomposition of inter-request versus intra-request cache hits across varying iteration depths of requests. At depth 0, cache hits are predominantly inter-request as requests share system prompt prefixes. SUTRADHARA’s priority policy explicitly prevents these prefixes from eviction leading to higher cache hits. As iteration depth increases, context grows with accumulation of previous iterations’ tool call outputs. SUTRADHARA increases intra-request hits as partial prefills issued by SUTRADHARA contains tool call outputs from previous iterations and are given least eviction priority. Overall, SUTRADHARA improves the global cache hit rate from 21.8% to 44.6%.

6 Comparison with Concurrent Works

Figure 12 compares SUTRADHARA against Continuum [11] configured with TTL = 6s (average tool execution time in production trace). SUTRADHARA reduces median FTR by 17% over Continuum. Continuum addresses only the cache thrashing bottleneck and does not overlap tool execution with prefill computation nor dispatches tools during decode. The execution in Continuum remains sequential regardless of TTL configuration. Furthermore, TTL-based pinning is sensitive to tool execution variance. A higher variance leads to misestimations of optimal TTL which mitigates benefits of KV pinning. Our trace shows a high variation in tool call times (§3), meaning a fixed TTL of 6s may cause cache evictions leading to higher tail latency. SUTRADHARA’s semantically aware priority based cache eviction policy remains effective regardless of tool execution variance.

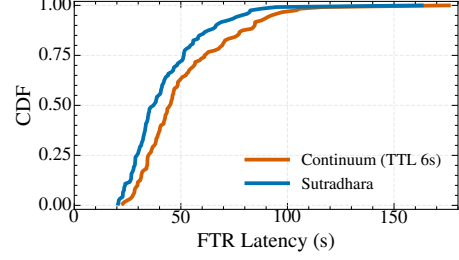


Figure 12. FTR-latency CDF for SUTRADHARA and Continuum.

7 Related Work

Table 3 compares SUTRADHARA with related works.

Agent Development. Several frameworks, such as Langraph [1], Autogen [18], Dspy [25], and Palimpsest [14], support agentic development by providing orchestration strategies for different applications. However, these frameworks treat the LLM backend as a black box and focus solely on orchestrator design. In contrast, our proposed interface enables co-design between the orchestrator and the LLM backend, which can be leveraged by these frameworks to improve system efficiency.

Agentic System Optimization. Prior works have explored optimizing agentic inference, but they differ fundamentally from SUTRADHARA in scope and assumptions. Parrot [12] and Murakkab [4] introduce declarative APIs to represent agentic workflows as static DAGs, allowing the LLM backend to optimize throughput-latency trade-offs using DAG information. In contrast, the tool-based agentic applications targeted by SUTRADHARA are not statically defined, and their DAG structure is unknown beforehand, making such approaches inapplicable. Autellix [16] proposes a non-clairvoyant scheduler to improve throughput, while KVFlow [21] focuses on KV-cache management for multi-agent applications. Circinus [13] introduces SLO-aware query planning to improve goodput, and DroidSpeak [15] explores KV-cache sharing across agents in multi-agent orchestration. However, these systems assume that agentic latency is not significantly influenced by tool calls—an assumption that does not hold in production, where tool calls are prevalent and contribute substantially to latency. SUTRADHARA addresses this gap by making agentic inference tool-aware through orchestrator-LLM backend co-design. Continuum [11] seeks to optimize KV cache management by predicting tool invocation times; however, our analysis reveals that these times exhibit substantial variability across requests, rendering accurate prediction impractical in our context. The closest work, Conveyor [28], overlaps partial tool execution with decoding. However, partial execution is infeasible for the agentic applications considered in the paper, as many tools cannot execute with incomplete parameters. Instead, SUTRADHARA tackles the practical challenge of handling tool-call fan-out, overlapping them with decoding.

Table 3. Comparison of related works against SUTRADHARA. ✓-supported; ✓-limited support; ✗-absent

Works	Dynamic Tool-Based Agents	Workload-Aware KV Cache Management	Tool-Decode Execution Overlap	Prefill-Tool Overlap
Parrot [12]	✗	✗	✗	✗
Murakkab [4]	✗	✗	✗	✗
Autellix [16]	✓	✗	✗	✗
Continuum [11]	✓	✓	✗	✗
Conveyor [28]	✓	✗	✓	✗
SUTRADHARA (Ours)	✓	✓	✓	✓

8 Conclusion

We present an extensive analysis of production tool-based LLM agentic applications, identifying three fundamental bottlenecks: (i) tool latency dominates end-to-end response time (30–85%), (ii) KV-cache effectiveness degrades despite significant context reuse across iterations, and (iii) sequential orchestration under-utilizes intra-request parallelism. These inefficiencies stem from decoupling orchestration from the LLM backend. We address this with SUTRADHARA, a co-designed agentic serving system. SUTRADHARA enables prompt-level parallelism, overlaps tool-call fan-outs with decoding, and incorporates workload-aware KV-cache management and scheduling. SUTRADHARA sustains up to 77% higher load at the same median latency, highlighting the need for coordinated orchestration–LLM co-design to scale agentic systems efficiently.

References

- [1] [n.d.]. GitHub - langchain-ai/langgraph: Build resilient language agents as graphs. — github.com. <https://github.com/langchain-ai/langgraph>. [Accessed 11-12-2025].
- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, 117–134. <https://www.usenix.org/conference/osdi24/presentation/agrawal>
- [3] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. 2022. DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [4] Gohar Irfan Chaudhry, Esha Choukse, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Adam Belay, and Ricardo Bianchini. 2025. Murakkab: Resource-Efficient Agentic Workflow Orchestration in Cloud Platforms. arXiv:2508.18298 [cs.MA] <https://arxiv.org/abs/2508.18298>
- [5] Yihua Cheng, Yuhang Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. 2025. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. *CoRR* abs/2510.09665 (2025). arXiv:2510.09665 doi:10.48550/ARXIV.2510.09665
- [6] An Yang et al. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [7] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQM66>
- [8] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *International Conference on Machine Learning (ICML)*.
- [9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 611–626. doi:10.1145/3600066.3613165
- [10] LangChain AI. 2025. LangChain — langchain.com. <https://www.langchain.com/>. [Accessed 11-12-2025].
- [11] Hanchen Li, Qiuyang Mang, Runyuan He, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230 [cs.OS] <https://arxiv.org/abs/2511.02230>
- [12] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA. <https://www.usenix.org/conference/osdi24/presentation/lin-chaofan>
- [13] Banruo Liu, Wei-Yu Lin, Minghao Fang, Yihan Jiang, and Fan Lai. 2025. Circinus: Efficient Query Planner for Compound ML Serving. arXiv:2504.16397 [cs.DB] <https://arxiv.org/abs/2504.16397>
- [14] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, and Gerardo Vitagliano. 2025. Palimpsest: Optimizing AI-Powered Analytics with Declarative Query Processing. In *Proceedings of the Conference on Innovative Database Research (CIDR)*.
- [15] Yuhang Liu, Yuyang Huang, Jiayi Yao, Shaoting Feng, Zhuohan Gu, Kuntai Du, Hanchen Li, Yihua Cheng, Junchen Jiang, Shan Lu, Madan Musuvathi, and Esha Choukse. 2025. DroidSpeak: KV Cache Sharing for Cross-LLM Communication and Multi-LLM Serving. arXiv:2411.02820 [cs.MA] <https://arxiv.org/abs/2411.02820>
- [16] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. 2025. Autellix: An Efficient Serving Engine for LLM Agents as General Programs. arXiv:2502.13965 [cs.LG] <https://arxiv.org/abs/2502.13965>
- [17] Baolin Miao, Yuntao Zhuang, Haichao Cui, Xupeng Zhang, Yang Yang, Zekai Wang, Pengcheng Li, Guodong Ding, Binhang He, Tianchi Chen, et al. 2024. LLM Inference Serving: Survey of Recent Advances and Opportunities. *arXiv preprint arXiv:2407.12391* (2024).
- [18] Microsoft. 2025. GitHub - microsoft/autogen: A programming framework for agentic AI — github.com. <https://github.com/microsoft/autogen>. [Accessed 11-12-2025].
- [19] Microsoft Azure. 2024. Azure VM NDm-A100-v4 sizes series. <https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/gpu-accelerated/ndma100v4-series>.
- [20] NVIDIA. 2024. NVIDIA TensorRT. <https://github.com/NVIDIA/TensorRT>.
- [21] Zaifeng Pan, Ajikumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. arXiv:2507.07400 [cs.DC] <https://arxiv.org/abs/2507.07400>

- [22] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large Language Model Connected with Massive APIs. *arXiv preprint arXiv:2305.15334* (2023).
- [23] Changle Qin, Aojun Zhang, Zihan Zhang, Jiaqi Chen, Michihiro Yasunaga, and Diyi Yang. 2024. Tool learning with large language models: a survey. *Frontiers of Computer Science* 18, 6 (2024).
- [24] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation - A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies, FAST 2025, Santa Clara, CA, February 25-27, 2025*, Haryadi S. Gunawi and Vasily Tarasov (Eds.). USENIX Association, 155–170. <https://www.usenix.org/conference/fast25/presentation/qin>
- [25] Stanford. [n.d.]. GitHub - stanfordnlp/dspy: DSPy: The framework for programming—not prompting—language models — github.com. <https://github.com/stanfordnlp/dspy>. [Accessed 11-12-2025].
- [26] Gemma Team. 2025. Gemma 3 Technical Report. arXiv:2503.19786 [cs.CL] <https://arxiv.org/abs/2503.19786>
- [27] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
- [28] Yechen Xu, Xinhao Kong, Tingjun Chen, and Danyang Zhuo. 2024. Conveyor: Efficient Tool-aware LLM Serving with Tool Partial Execution. <https://openreview.net/forum?id=A0VvDN4arV>
- [29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).
- [30] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, Clark Barrett, and Ying Ying. 2023. SGLang: Efficient Execution of Structured Language Model Programs. *arXiv preprint arXiv:2312.07104* (2023).

A Appendix

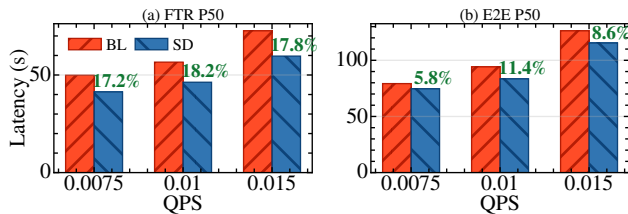


Figure A.2. Median FTR and E2E latency across QPS levels on a another subset of the production trace. SUTRADHARA consistently reduces FTR by 17–18% and E2E by 6–11% across all load levels.

A.1 Detailed Latency CDFs on Production Trace

Figure A.1 shows the full FTR and E2E latency CDFs for SUTRADHARA and the baseline across three load levels, evaluated on a subset of the top 120 high fan-out requests from the production trace. At 0.0075 and 0.01 QPS, SUTRADHARA consistently shifts the CDF left across the entire distribution, yielding median improvements of 15–16% in FTR and 9–10% in E2E. Furthermore, at 0.0125 QPS—which is 25% beyond the

baseline’s maximum sustainable load SUTRADHARA achieves a lower median FTR (37s versus 40s) and E2E (66s versus 67s) than the baseline operating at the strictly lower 0.01 QPS. This directly confirms the 25% higher serving capacity reported in §5.2.

A.2 Robustness Across Production Trace Subsets

Figure A.2 validates that SUTRADHARA’s gains are not artifacts of the high fan-out subset used in the primary evaluation. We evaluate a separate 60-request subset sampled to represent the median tool call fan-out of the same production trace. Across all load levels, SUTRADHARA consistently reduces FTR P50 by 17–18% and E2E P50 by 6–11%, closely matching the performance gains reported on the primary trace. This confirms that SUTRADHARA’s architectural optimizations remain robust across diverse operating regimes within the production workload distribution.

A.3 Limitations of Orchestrator-Engine Coupling

Frameworks like LangGraph [1] are well positioned to adopt SUTRADHARA’s interface. LangGraph’s graph-based execution model exposes explicit boundaries between LLMs and tool execution as discrete named nodes. The key adaptation is extending the LLM node to support concurrent operations during decode i.e eagerly submitting tool-independent prompt content and dispatching individual tools as their specifications complete, rather than treating decode as a strictly sequential, atomic step. These changes are confined to the LLM node and the interface layer between the orchestrator and the engine, leaving LangGraph’s core execution model intact. We expect the same interface to generalize to other graph-based frameworks such as AutoGen [18] as they adopt similar execution models.

While SUTRADHARA’s co-design of the orchestrator and serving engine is strictly necessary to enable performance optimizations like streaming dispatch and orchestrator-aware KV cache pinning, it inherently trades deployment flexibility for execution efficiency. Specifically, SUTRADHARA’s extended five-API interface introduces a tight versioning dependency between the orchestrator and the LLM serving engine. Upgrading either component requires verifying API compatibility, adding operational overhead compared to traditional black-box deployments where the two layers evolve independently. Consequently, organizations adopting SUTRADHARA must coordinate releases across both layers rather than treating the inference engine as a modular, drop-in replacement.

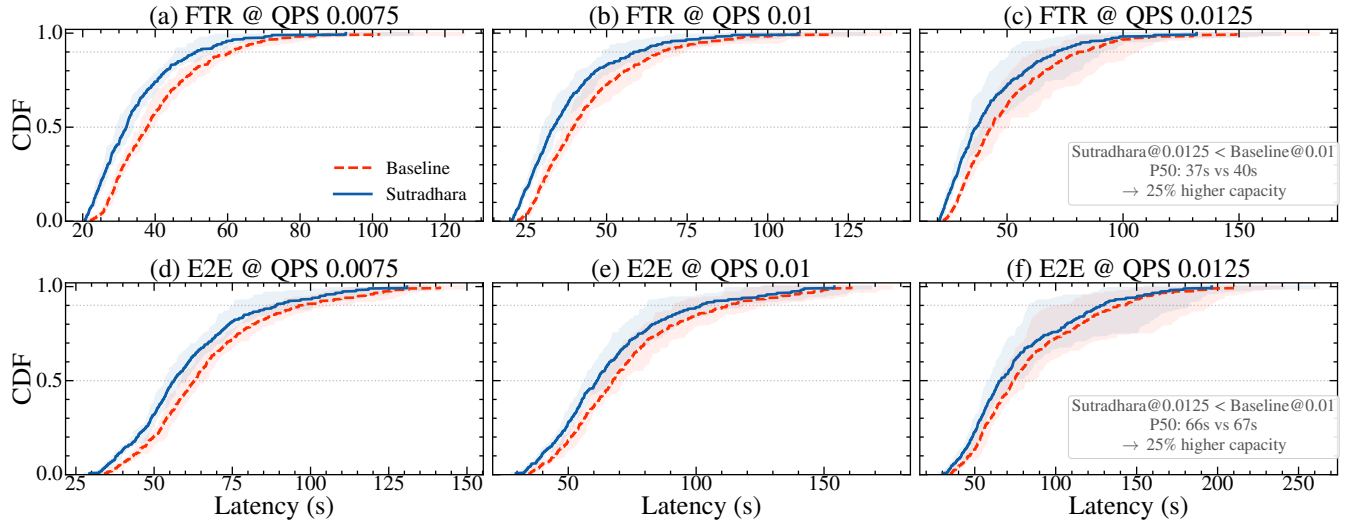


Figure A.1. Comparison of FTR and E2E Latency on the sampled Production trace